Project Report: Scientific Calculator (DEVOPS)

Submitted by: Siddhesh Sandeep Mahajan
Roll Number: MT2025122
Date: March 7, 2026

1. Introduction: What and Why of DevOps?

What is DevOps?

DevOps is a set of practices that combines Software Development (Dev) and IT Operations (Ops). It aims to shorten the systems development life cycle and provide continuous delivery with high software quality.

Why DevOps?

Speed: Automates testing and deployment to deliver features faster.

Reliability: Ensures consistent environments from development to production.

Collaboration: Breaks down silos between developers and system administrators.

2. Tools Used

Version Control: Git & GitHub

Automation Server: Jenkins

Containerization: Docker & Docker Hub

Configuration Management: Ansible

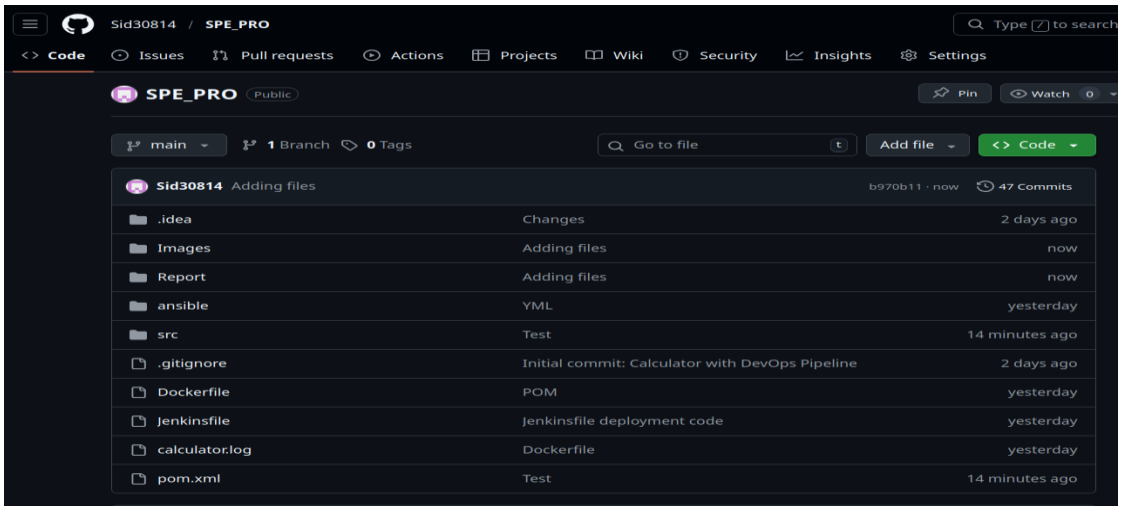
JUnit4 : Unit testing framework

3. Project Steps & Implementation

Step 1: Source Control Management (GitHub)

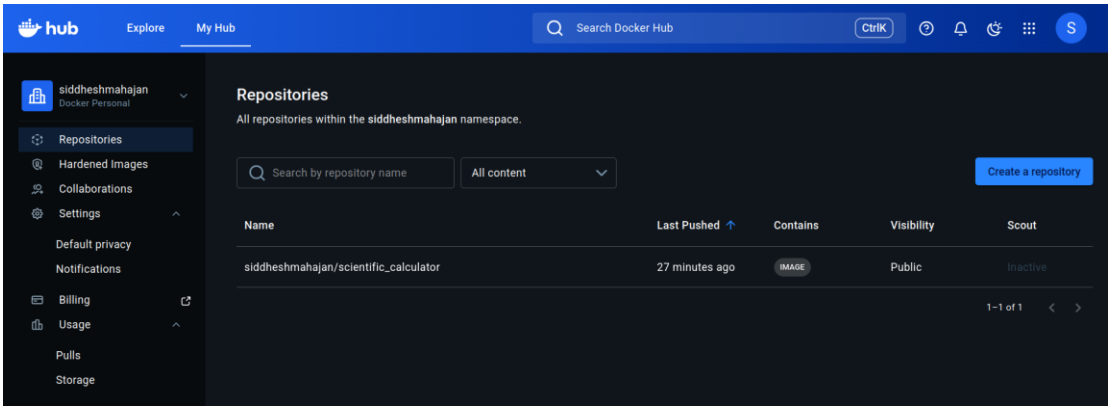
Brief: We used Git for version control and GitHub to host our remote repository.

Setup: Created a new repository named SPE_PRO.



Step 2: Containerization (Docker)

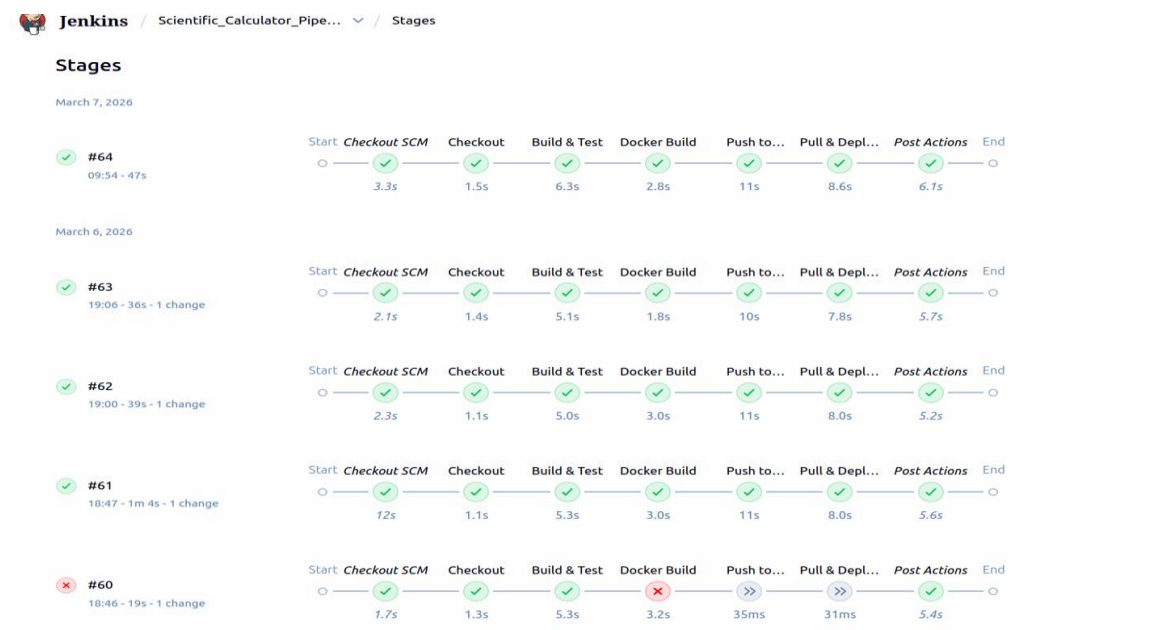
Brief: Docker allows us to package the application and its dependencies into an immutable image.



Step 3: CI/CD Pipeline (Jenkins)

Brief: Jenkins serves as the orchestration layer to automate the entire process.

Setup: Configured a Pipeline job and added Docker Hub credentials in Jenkins Manage Credentials.



Step 4: Deployment Automation (Ansible)

Brief: Ansible automates server configuration and container deployment without needing manual SSH commands.

sh 'ansible-playbook -i ansible/inventory.ini ansible/playbook_pull.yml'

```
siddhesh-mahajan@siddhesh-mahajan-Vivobook-ASUSLaptop-X1605VA-X1605VA:~/Downloads/SPE_PRO$ docker logs calculator_service
Mar 07, 2026 4:25:34 AM org.example.ScientificCalculator main
INFO: Application started.

--- Scientific Calculator (CS 816) ---
1. Square root function (√x)
2. Factorial function (x!)
3. Natural logarithm (ln(x))
4. Power function (x^b)
5. Exit
```

```
siddhesh-mahajan@siddhesh-mahajan-Vivobook-ASUSLaptop-X1605VA-X1605VA:~/Download
s/SPE_PRO$ docker ps
CONTAINER ID   IMAGE                                          COMMAND
       CREATED          STATUS          PORTS     NAMES
f4162d82a4a8   siddheshmahajan/scientific_calculator:latest   "java -jar /app/sc
ie…"   17 seconds ago   Up 16 seconds             calculator_service
```

Step 5. Final Verification

```
siddhesh-mahajan@siddhesh-mahajan-Vivobook-ASUSLaptop-X1605VA-X1605VA:~/Downloads/SPE_PRO$ docker exec -it calculator_service java -jar /app/scientific-c
alculator.jar
Mar 07, 2026 4:38:21 AM org.example.ScientificCalculator main
INFO: Application started.

--- Scientific Calculator (CS 816) ---
1. Square root function (√x)
2. Factorial function (x!)
3. Natural logarithm (ln(x))
4. Power function (x^b)
5. Exit
Enter your choice: 1
Enter x: 4
Result: 2.0
Mar 07, 2026 4:38:30 AM org.example.ScientificCalculator main
INFO: Square Root of 4.0 = 2.0
```

Project Phases

Phase 1: Preparation and Structuring

The project begins by organizing the source code into a standardized layout. We ensure the main class is correctly placed within the project's directory tree, allowing automated tools to identify the application's entry point.

Jenkins Stage: Checkout

This step establishes a connection to the GitHub repository and clones the main branch.

stage('Checkout') { steps { git branch: 'main', url: 'https://github.com/Sid30814/SPE_PRO.git' } }

Phase 2: Building and Packaging

We transform the raw code into a ready-to-run package by compiling the files and resolving dependencies. We configure build instructions to specify the starting class, ensuring the Java engine knows exactly which program to launch.

Jenkins Stage: Build & Test

This stage cleans the environment, compiles source code, and runs the JUnit test suite.

stage('Build & Test') { steps { sh 'mvn clean package' } }

```
 T E S T S
-------------------------------------------------------
Running org.example.ScientificCalculatorTest
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.069 sec

Results :

Tests run: 3, Failures: 0, Errors: 0, Skipped: 0

[INFO]
[INFO] --- maven-jar-plugin:3.3.0:jar (default-jar) @ ScientificCalculator ---
[INFO] Building jar: /home/siddhesh-mahajan/Downloads/SPE_PRO/target/ScientificC
alculator-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  2.318 s
[INFO] Finished at: 2026-03-07T22:24:14+05:30
[INFO] ------------------------------------------------------------------------
```

Phase 3: Containerization

We create a blueprint for a small, isolated environment containing only the software needed to run the calculator application. This portable image ensures the application works identically on any machine.

Jenkins Stages: Docker Build & Push

These stages construct the Docker image based on the Dockerfile and securely push it to the Docker Hub registry.

stage('Docker Build') { steps { sh "docker build -t ${DOCKER_HUB_USER}/${IMAGE_NAME}:latest ." } }
stage('Push to Docker Hub') { steps { withCredentials(...) { sh "docker push ${DOCKER_HUB_USER}/${IMAGE_NAME}:latest" } } }

Phase 4: Automation and Deployment

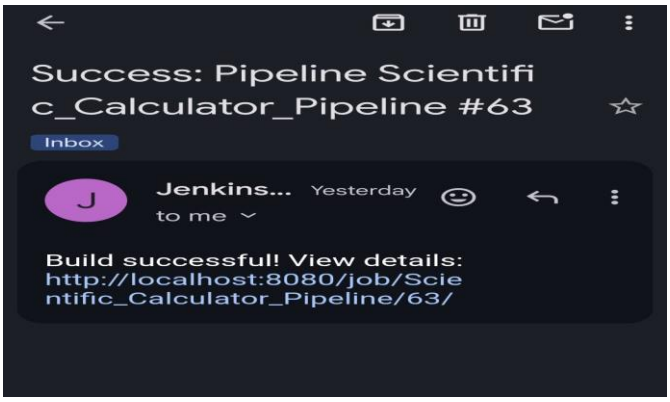
An orchestration tool is used to instruct the server to fetch the latest application image from the Docker Hub registry. The system identifies the existing container, gracefully stops it to free the network port, and launches the updated version of the application.

Jenkins Stage: Pull & Deploy

This stage executes the Ansible playbook, which manages server-side deployment and restarts the application container.

stage('Pull & Deploy') { steps { sh 'ansible-playbook -i ansible/inventory.ini ansible/playbook_pull.yml' } }

In the last phase Jenkins alerts the email id recipient about the build status.



WHY Ngrok used

In this project, **ngrok** serves as the vital link between our private development environment and the public internet. Since your local Jenkins server resides behind a firewall and lacks a public IP address, external services like GitHub cannot "see" it to send automated notifications. By initiating a secure tunnel, ngrok provides a temporary public URL that forwards traffic directly to your local machine. This allows GitHub Webhooks to instantly trigger your pipeline the moment code is pushed, transforming a manual build process into a truly automated, event-driven CI/CD workflow.

4. Project Links

GitHub Repository: [https://github.com/Sid30814/SPE_PRO.git]

Docker Hub Repository:
[https://hub.docker.com/r/siddheshmahajan/scientific_calculator/tags]